Async Python

...

The (bright) future after the end of GIL

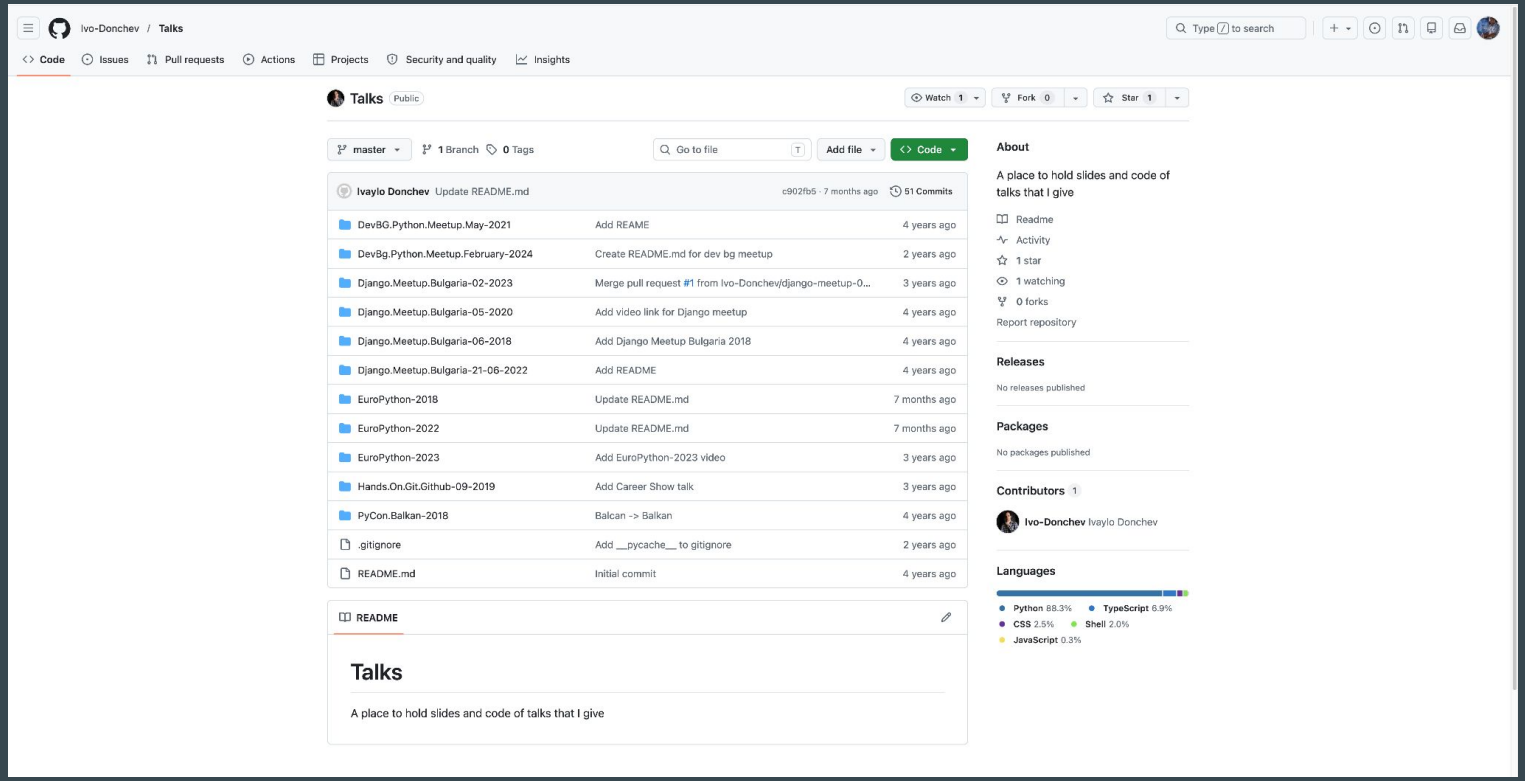
About Me

Ivaylo Donchev

Senior Full-Stack Developer / Technical Team Lead @ Waracle

 Ivo-Donchev

 in/ivaylo-donchev



Source: https://github.com/Ivo-Donchev/Talks

Motivation

PEP 734 – Multiple Interpreters in the Stdlib

Author:	Eric Snow <ericsnowcurrently at gmail.com>
Discussions-To:	Discourse thread
Status:	Final
Type:	Standards Track
Created:	06-Nov-2023
Python-Version:	3.14
Post-History:	14-Dec-2023
Replaces:	554
Resolution:	05-Jun-2025

Source: https://peps.python.org/pep-0734/


django


FastAPI


Flask

Python is too slow

Performance is overrated



“Performance”

“GIL”

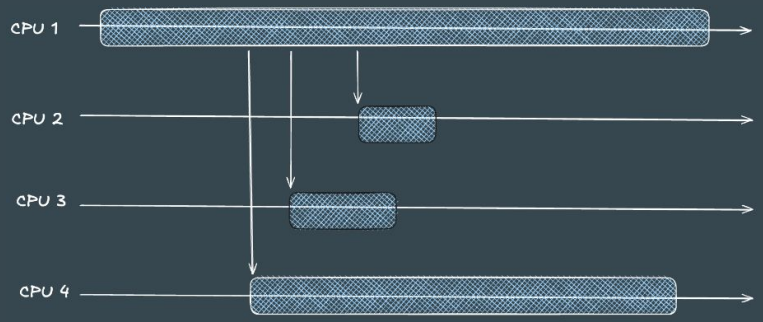
Global Interpreter Lock

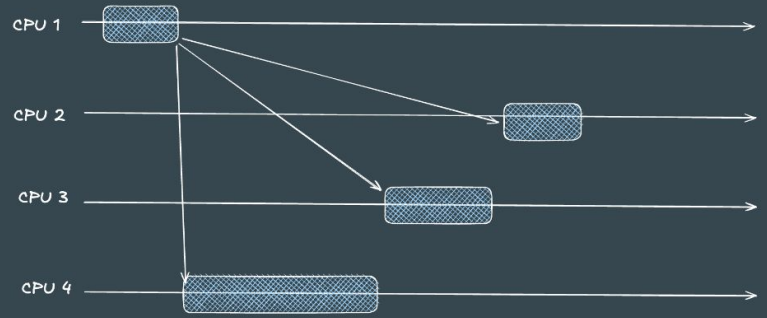


The "North Star" Questions:

- How does the async programming in python work?
- What is GIL?
- Why is it time to remove GIL?
- What is "thread-free" python?
- What should we expect in the future?

Async Python





Task Types in Python

I/O Bound Tasks

```
def send_invitation_emails():
    for user in users:
        send_invitation_email(user)
```

CPU-Bound Tasks

```
def fibonacci(n):
    if n <= 1:
        return n
    else:
        return (
            fibonacci(n - 1) +
            fibonacci(n - 2)
        )
```

Asynchronous programming in Python

- Process
- Thread
- Coroutine

Process



Parallelism

Allows using multiple cores



Overhead

Heavy to start compared to threads



Consumption

Uses a lot more memory



Isolation

Cannot reuse memory from other process

Process in Python

```python
import multiprocessing as mp
import os

def worker(i: int) -> None:
    print(f"Task {i} -> PID {os.getpid()}")

if __name__ == "__main__":
    print(f"Main PID: {os.getpid()}")
    procs = [
        mp.Process(target=worker, args=(i,))
        for i in range(4)
    ]
    for p in procs:
      p.start()
    for p in procs:
      p.join()
    print("Done")
```

Output

```
Main PID: 92186
Task 1 -> PID 92189
Task 0 -> PID 92188
Task 2 -> PID 92190
Task 3 -> PID 92191
Done
```

Thread



Lightweight

Faster to start than processes



Shared Memory

Communicate easily via shared state



Low Memory

Much lower overhead than processes



*I/O Efficiency

Ideal for waiting on network or disk

Python Threading Example

```python
import threading
import time

def worker(name: str, delay: float) -> None:
    for i in range(3):
        time.sleep(delay)
        print(f"{name}: step {i+1}")

if __name__ == "__main__":
    t1 = threading.Thread(
        target=worker, args=("Thread-A", 0.4)
    )
    t2 = threading.Thread(
        target=worker, args=("Thread-B", 0.6)
    )
    t1.start(); t2.start()
    t1.join(); t2.join()
    print("Done.")
```

Output

```
Thread-A: step 1
Thread-B: step 1
Thread-A: step 2
Thread-B: step 2
Thread-A: step 3
Thread-B: step 3

Done.
```

"thread" in Python



OS Threads

- Managed by Operating System
- True parallel execution on multi-core CPUs
- Context switching handled by kernel

!=



Python Threads

- Managed by Python Interpreter
- **Parallelism** only for I/O Bound tasks
- Restricted by the **GIL** for CPU bound tasks

tl;dr: The GIL ensures only one thread executes Python bytecode at a time.

Why?

The year is 1991...



Python Released

Guido van Rossum officially releases Python 0.9.0.



Async "Buzzword"

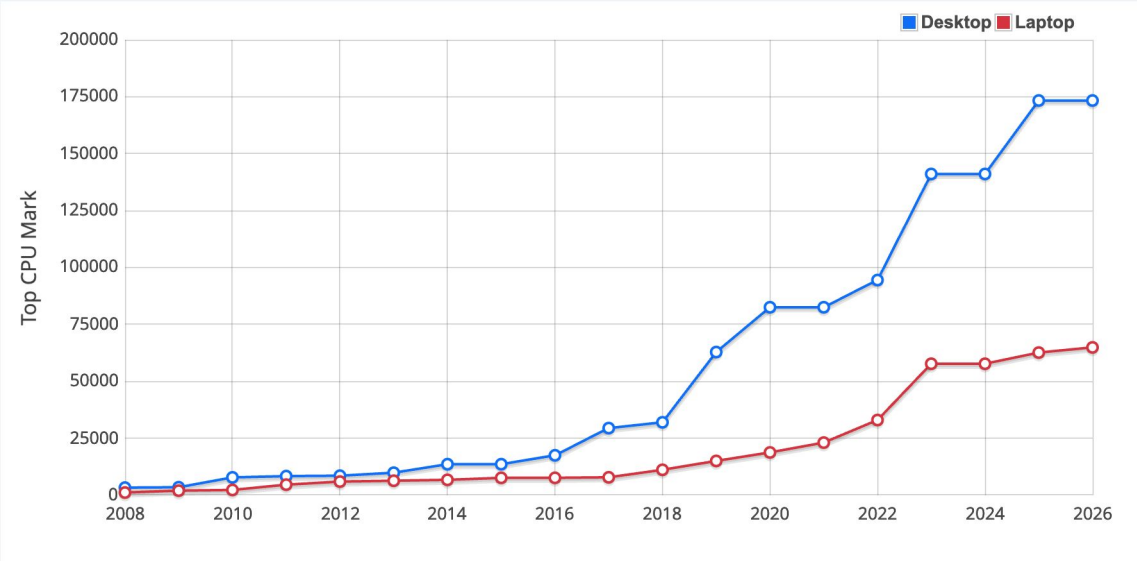
Asynchronous programming is a theoretical concept rather than a standard practice.



Hardware

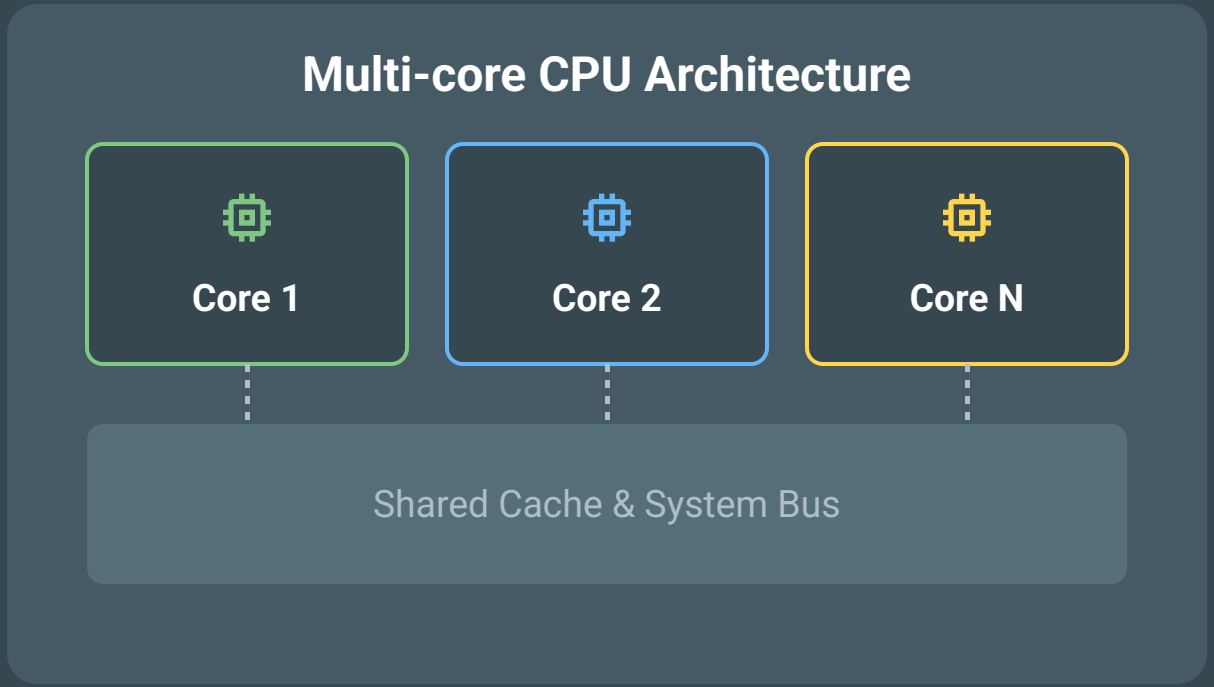
Single-core CPUs.

CPU vertical scaling is far from good..



Source: https://www.cpubenchmark.net/year-on-year.html

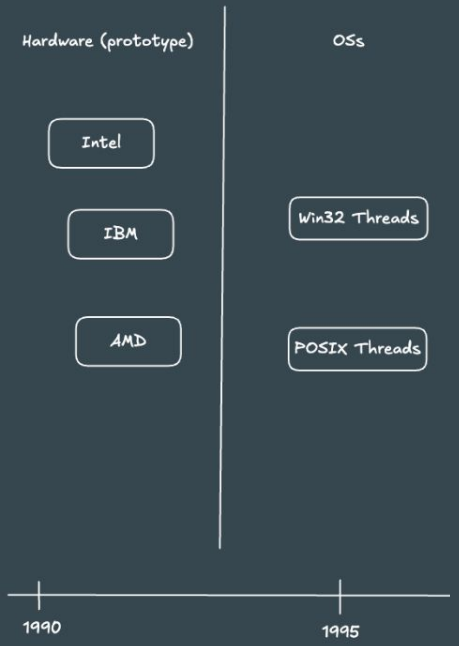
However, horizontal scaling is far more achievable...



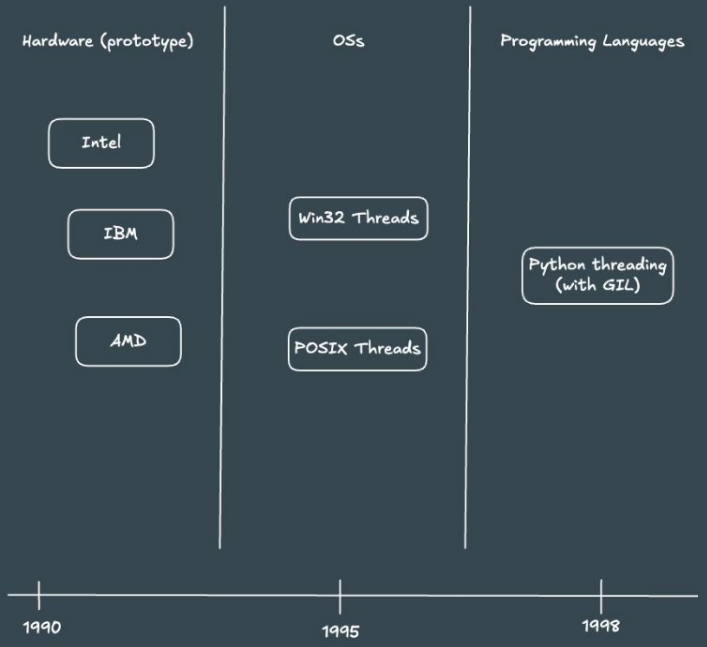
Threads history



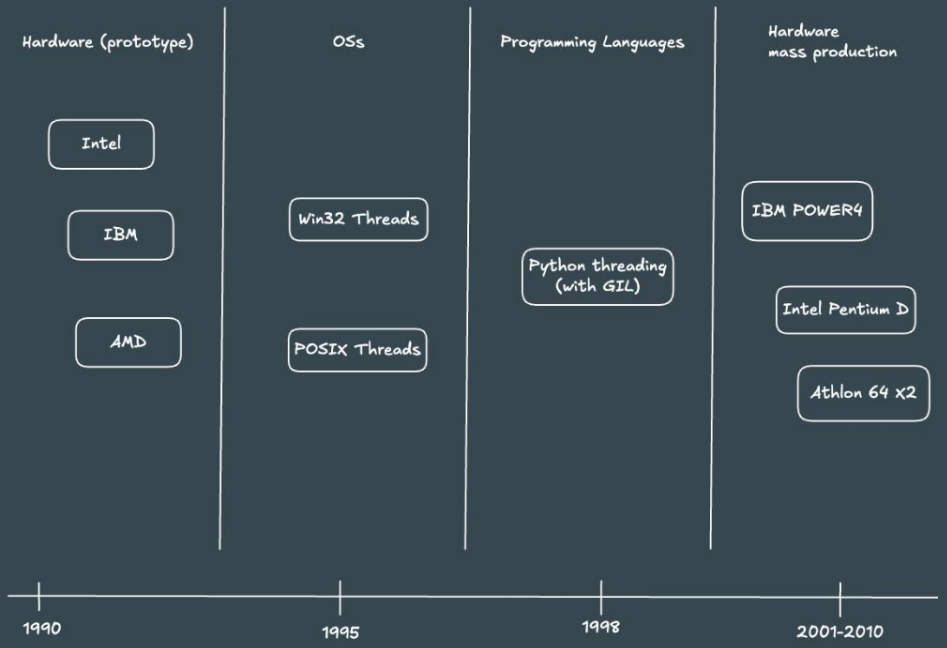
Threads history



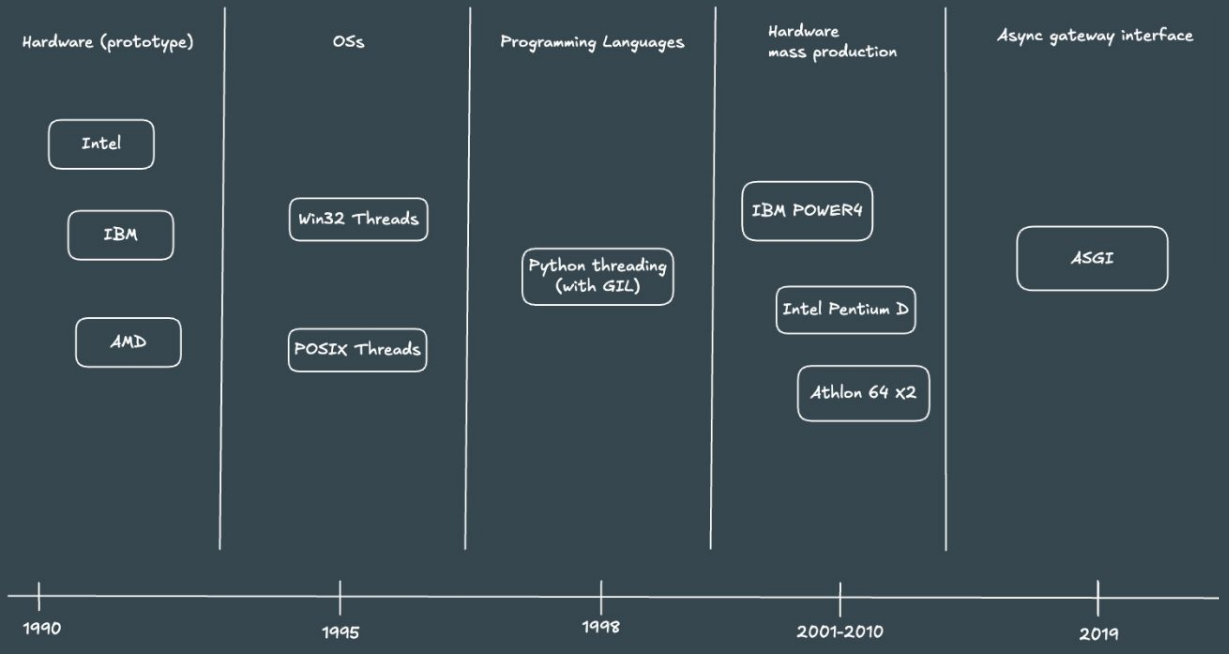
Threads history



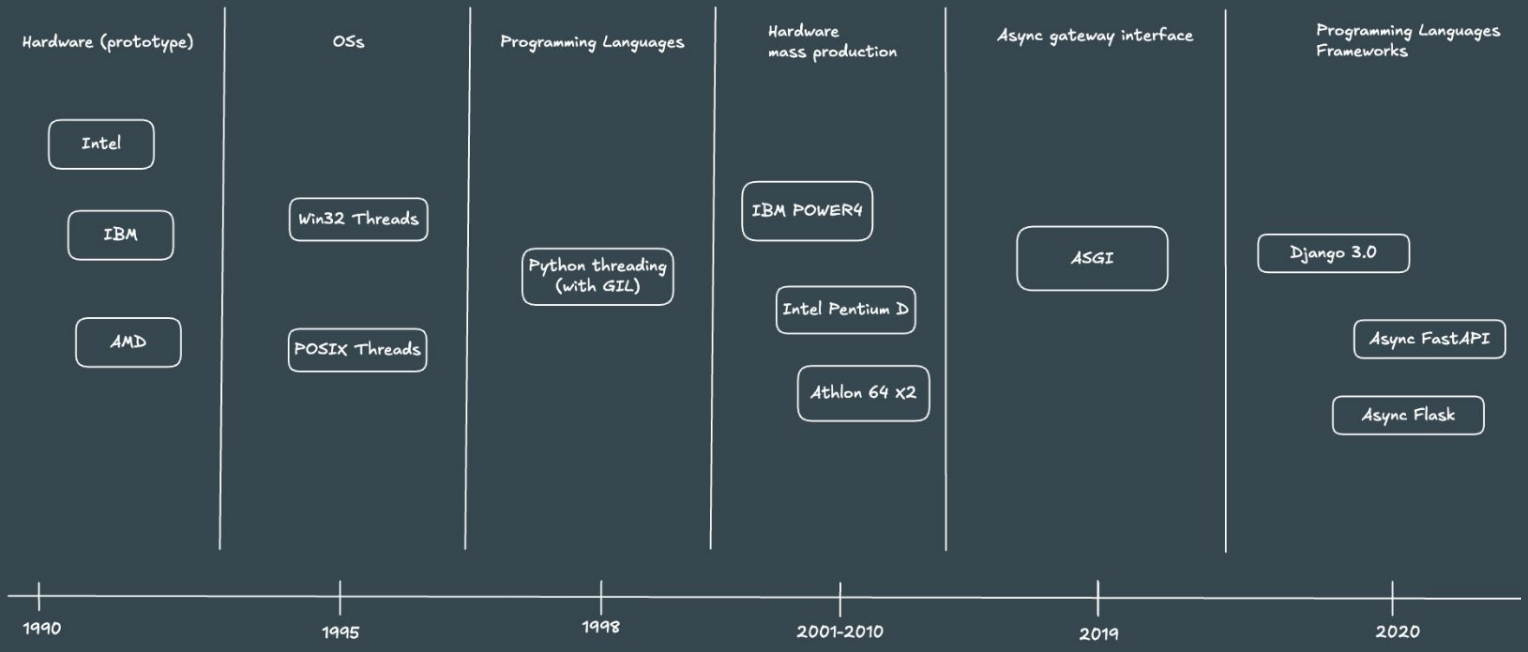
Threads history



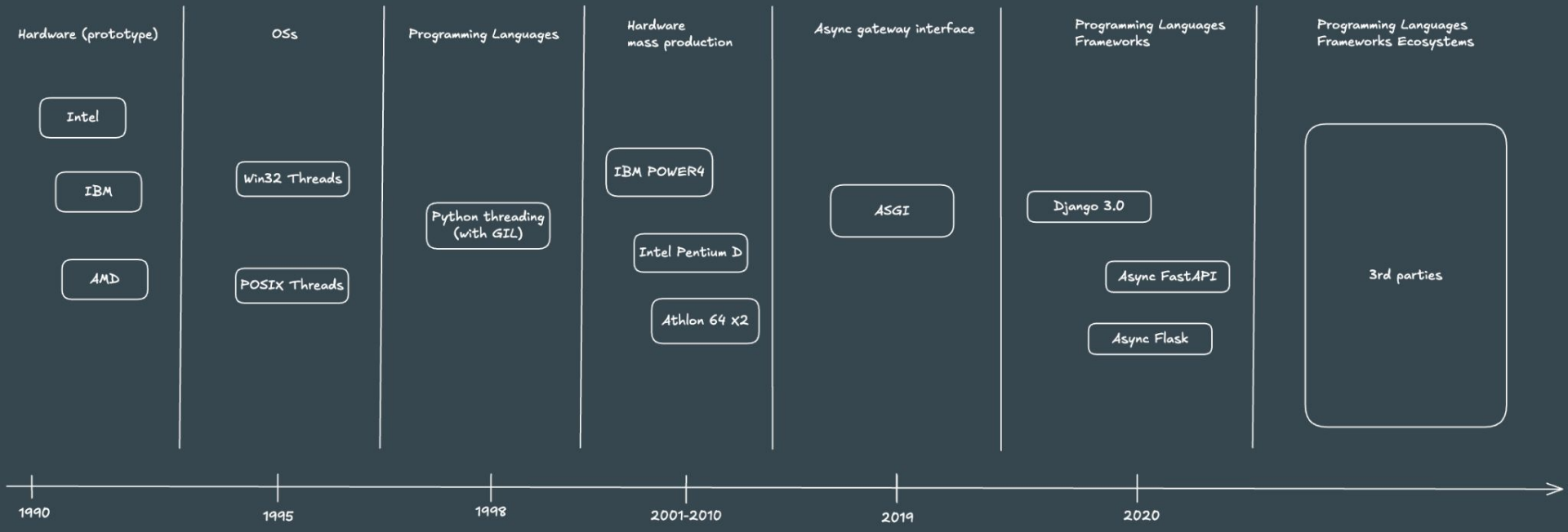
Threads history



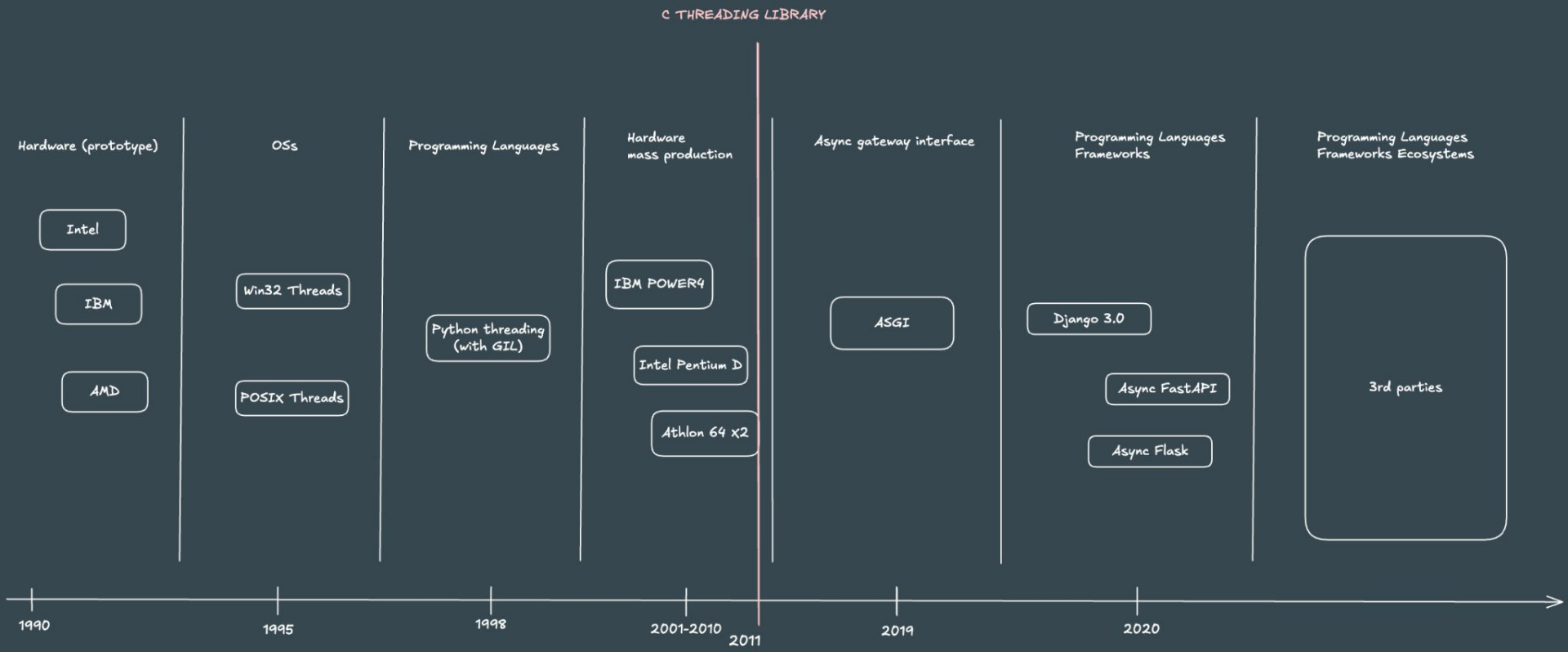
Threads history



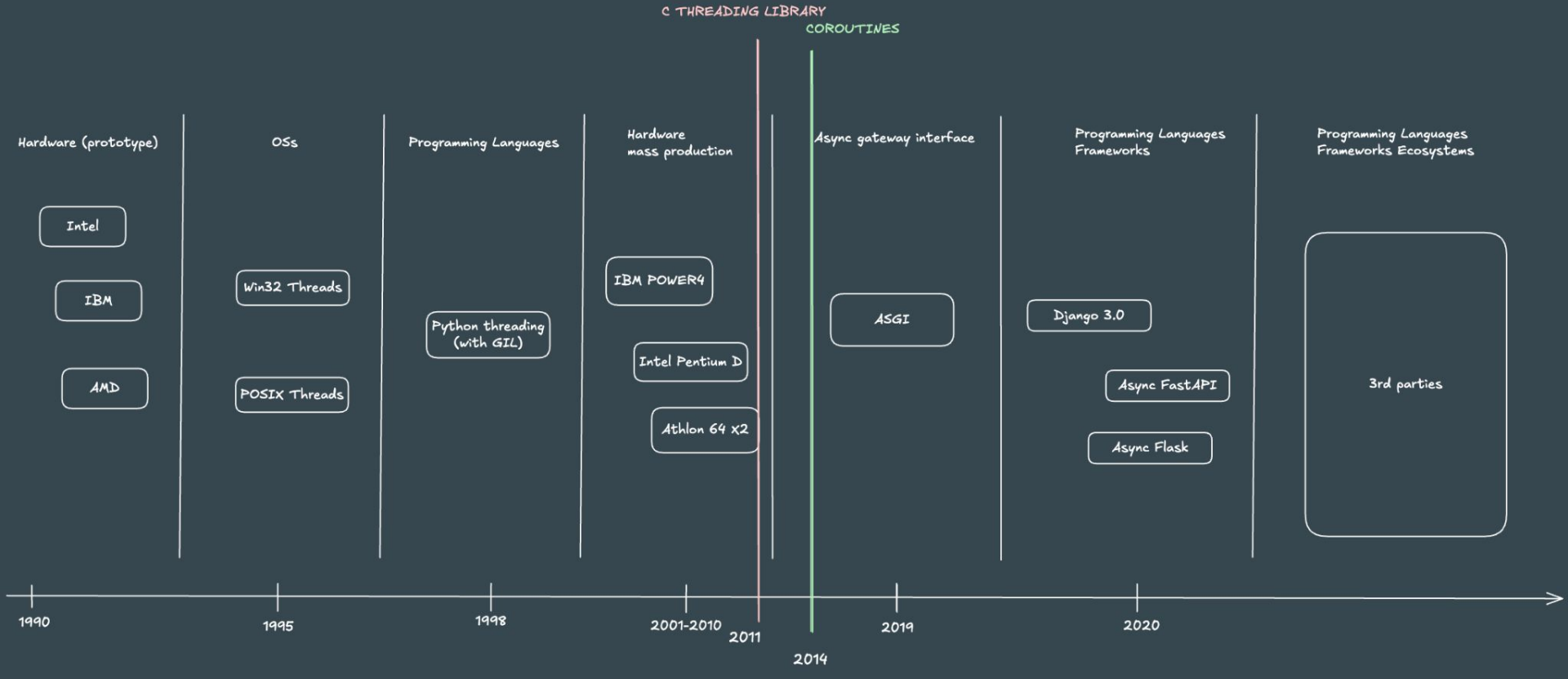
Threads history



Threads history



Threads history



THREADS DEMO
I/O vs CPU Bound tasks

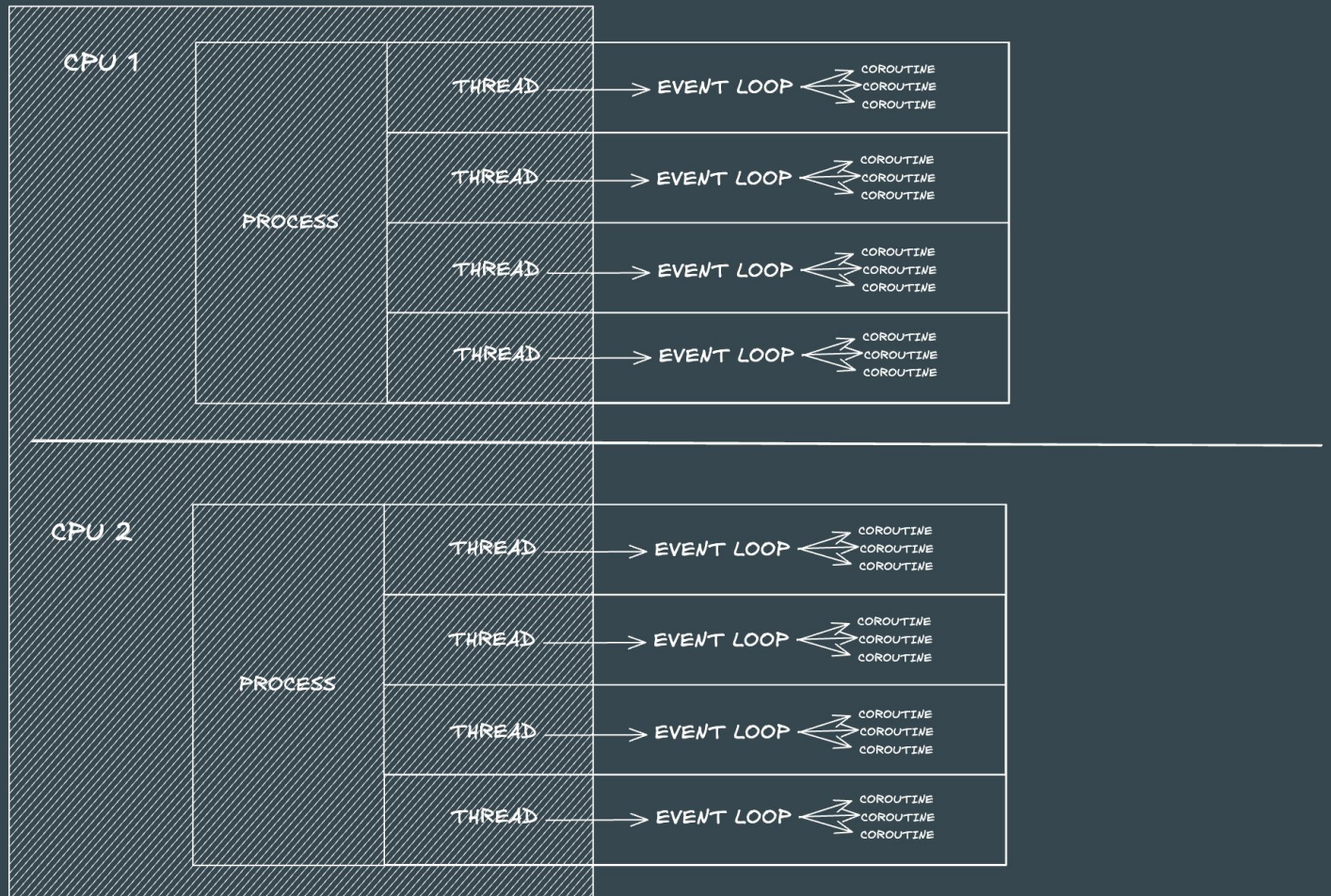
Coroutines

COROUTINES DEMO
I/O vs CPU Bound tasks

gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 2 --bind 0.0.0.0:8000



gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 2 --bind 0.0.0.0:8000






```python
class AsyncHelloView(APIView):
    async def get(self, request):
        return Response({"message": "Hello, async world!"})
```

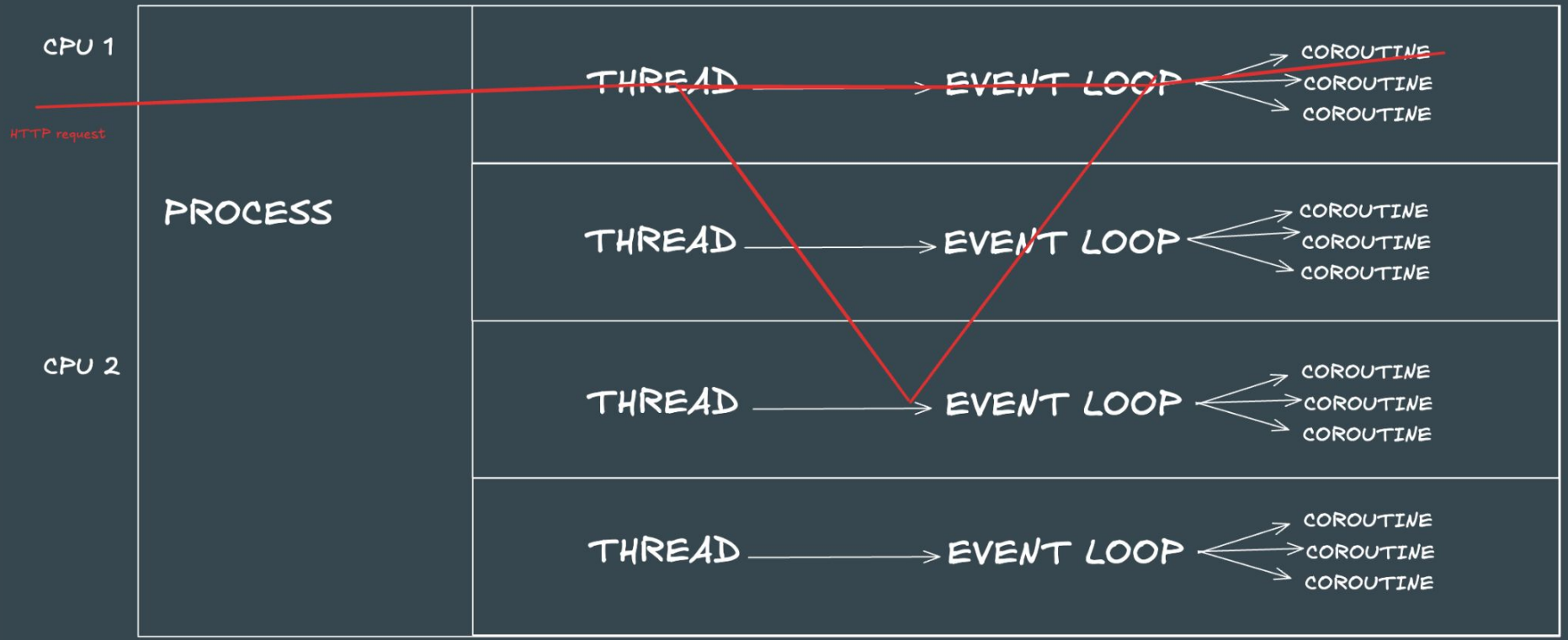
gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 2 --bind 0.0.0.0:8000



gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 1 --bind 0.0.0.0:8000



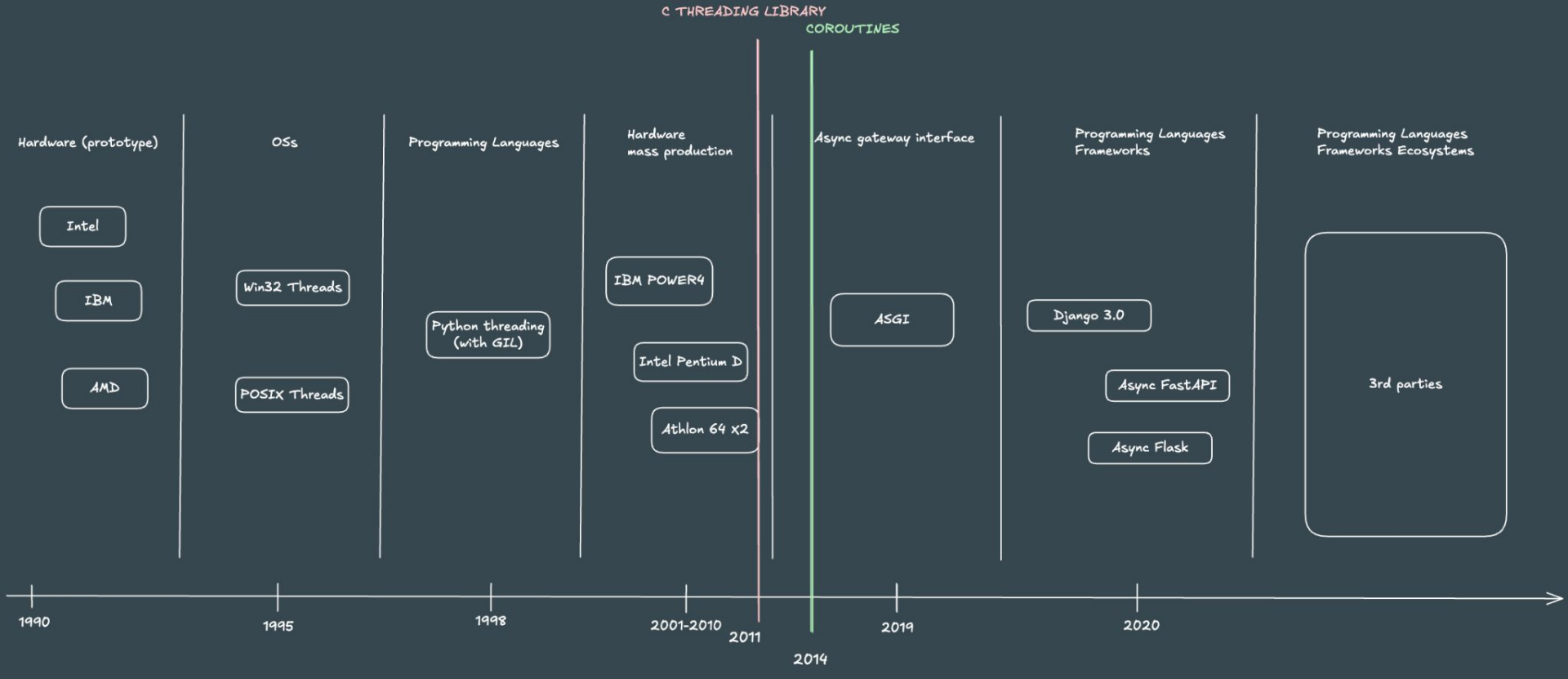
gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 1 --bind 0.0.0.0:8000



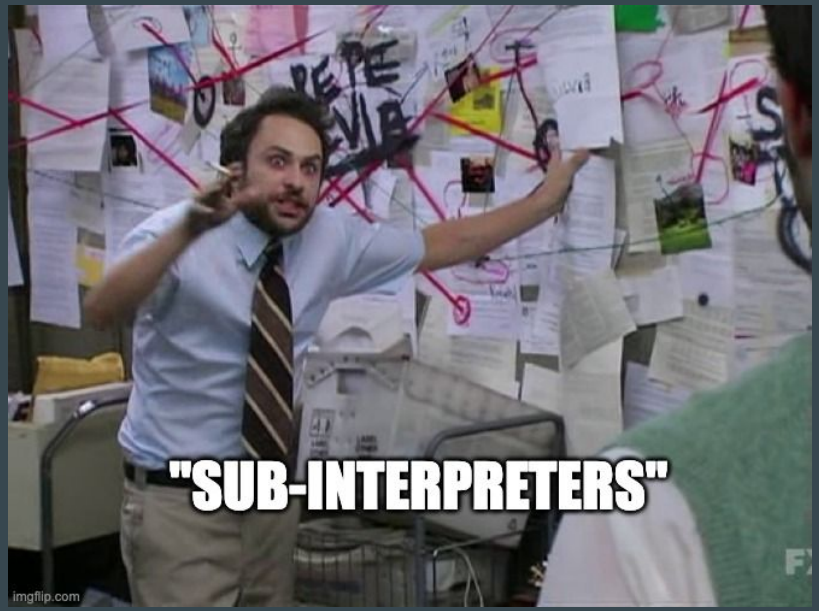
Future has no place for GIL

Solution #1
Refactor everything

Threads history



Solution #2
“Sub-interpreters”



Solution #2
Sub interpreters

PEP 734 – Multiple Interpreters in the Stdlib

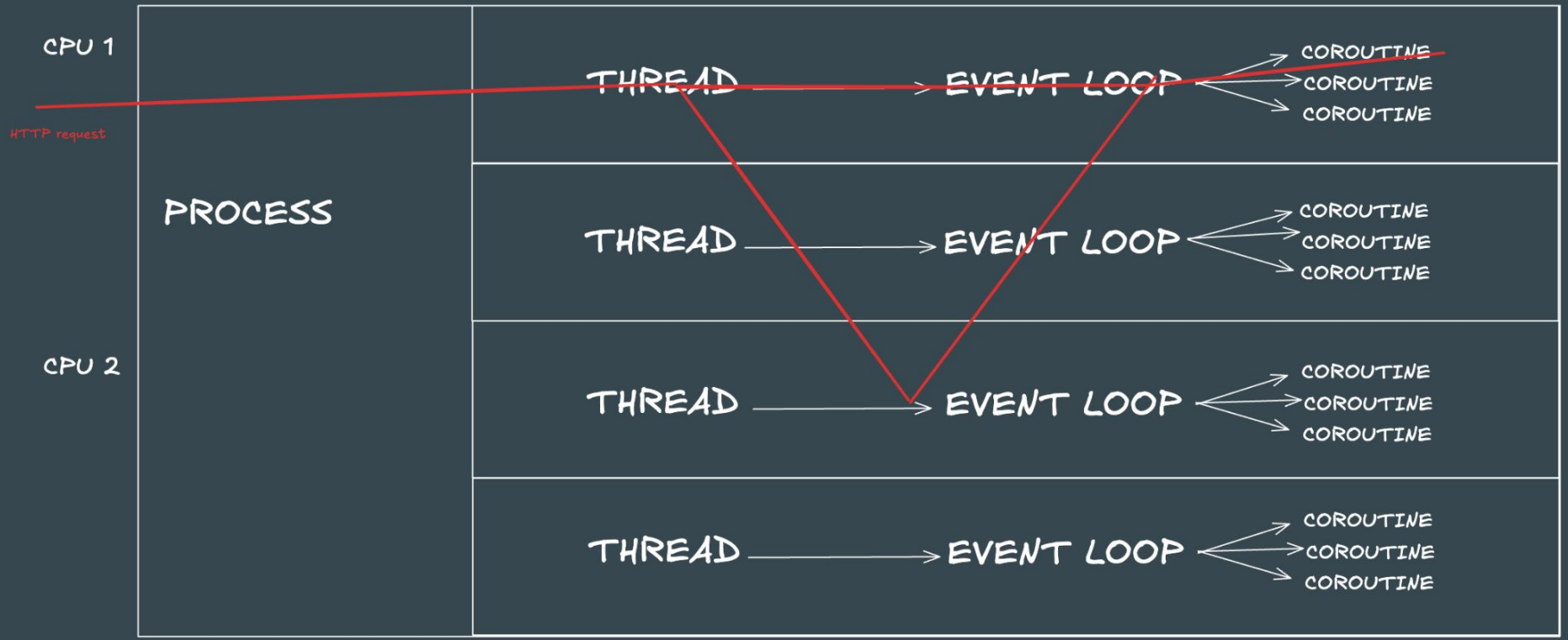
Author:	Eric Snow <ericsnowcurrently at gmail.com>
Discussions-To:	Discourse thread
Status:	Final
Type:	Standards Track
Created:	06-Nov-2023
Python-Version:	3.14
Post-History:	14-Dec-2023
Replaces:	554
Resolution:	05-Jun-2025

Sub-interpreters: Parallel Execution



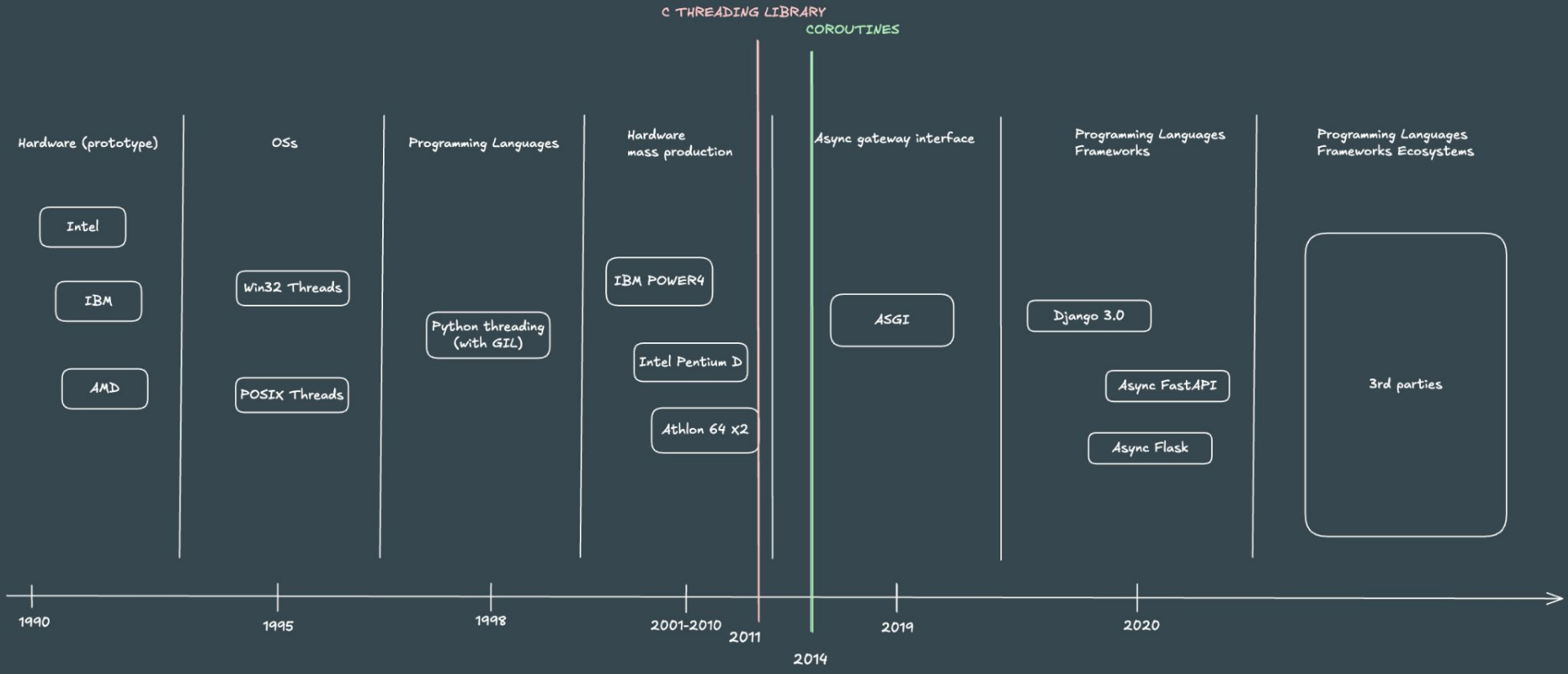
Each sub-interpreter has its own Global Interpreter Lock (GIL)

gunicorn myproject.asgi:application -k uvicorn.workers.UvicornWorker --workers 1 --bind 0.0.0.0:8000



DEMO #3: Coroutines + Threads

Threads history



Q&A


CPU 1
CPU 2
PROCESS
THREAD
EVENT LOOP
COROUTINE
COROUTINE
COROUTINE
THREAD
EVENT LOOP
COROUTINE
COROUTINE
COROUTINE
THREAD
EVENT LOOP
COROUTINE
COROUTINE
COROUTINE
THREAD
EVENT LOOP
COROUTINE
COROUTINE
COROUTINE